

PRACTICAL-4:

Aim: Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

PROCEDURE :

1. Create multiple child processes and compare wait() vs waitpid() :

```
siddinapujith07@DESKTOP-FdHCAIT:~$ mkdir practical4
siddinapujith07@DESKTOP-FdHCAIT:~$ cd practical4
siddinapujith07@DESKTOP-FdHCAIT:~/practical4$ nano wait_compare.c
siddinapujith07@DESKTOP-FdHCAIT:~/practical4$ gcc wait_compare.c -o wait_compare
siddinapujith07@DESKTOP-FdHCAIT:~/practical4$ ./wait_compare
Child 1: PID = 523
Child 2: PID = 524

Parent PID = 522

Using wait():
Child 3: PID = 525
Child 2 completed
wait() collected child PID = 524

Using waitpid():
Child 3 completed
Child 1 completed
waitpid() collected Child 1 PID = 523
waitpid() collected Child 2 PID = 524
waitpid() collected Child 3 PID = 525

All children completed.
```

2. Create a zombie process :

```
siddinapujith07@DESKTOP-FdHCAIT:~/practical4$ nano zombie.c
siddinapujith07@DESKTOP-FdHCAIT:~/practical4$ gcc zombie.c -o zombie
siddinapujith07@DESKTOP-FdHCAIT:~/practical4$ ./zombie
Parent process: PID = 565
Child PID = 566
Parent sleeping for 30 seconds...
Child process: PID = 566
Child terminating...
Parent terminating...
siddinapujith07@DESKTOP-FdHCAIT:~/practical4$ ps -o pid,ppid,state,stat,cmd -p 566
  PID   PPID  S   STAT  CMD
  565     56  S   Ss    sleep 30
  566     56  Z   Zz    ps -o pid,ppid,state,stat,cmd -p 566
```

3. Fixed zombie process :

```
siddinapujith07@DESKTOP-FdHCAIT:~/practical4$ nano zombie.c
siddinapujith07@DESKTOP-FdHCAIT:~/practical4$ gcc zombie.c -o zombie
siddinapujith07@DESKTOP-FdHCAIT:~/practical4$ ./zombie
Parent process: PID = 806
Child PID = 807
Child process: PID = 807
Child terminating...
Child collected successfully.
Parent terminating...
```

Code:

```
GNU nano 8.7.1      wait_compare.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t c1, c2, c3;
    int status;

    c1 = fork();
    if (c1 == 0)
    {
        printf("Child 1: PID = %d\n", getpid());
        sleep(3);
        printf("Child 1 completed\n");
        exit(1);
    }
    c2 = fork();

    if (c2 == 0)
    {
        printf("Child 2: PID = %d\n", getpid());
        sleep(1);
        printf("Child 2 completed\n");
        exit(2);
    }
    c3 = fork();

    if (c3 == 0)
    {
        printf("Child 3: PID = %d\n", getpid());
        sleep(2);
        printf("Child 3 completed\n");
        exit(3);
    }
}
```

```
printf("\nParent PID = %d\n", getpid());  
  
printf("\nUsing wait():\n");  
  
pid_t finished = wait(&status);  
  
printf("wait() collected child PID = %d\n", finished);  
  
printf("\nUsing waitpid():\n");  
  
waitpid(c1, &status, 0);  
printf("waitpid() collected Child 1 PID = %d\n", c1);  
  
waitpid(c2, &status, 0);  
printf("waitpid() collected Child 2 PID = %d\n", c2);  
  
waitpid(c3, &status, 0);  
printf("waitpid() collected Child 3 PID = %d\n", c3);  
  
printf("\nAll children completed.\n");  
  
return 0;
```

```
}
```

Zombie Code :

```
GNU nano 8.7.1          zombie.c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child terminating...\n");
        exit(0);
    }
    else
    {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        printf("Parent sleeping for 30 seconds...\n");
        sleep(30);

        printf("Parent terminating...\n");
    }

    return 0;
}
```

Fixed zombie Code :

```
GNU nano 8.7.1      zombie.c *
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child terminating...\n");
        exit(0);
    }
    else
    {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        waitpid(pid, NULL, 0);

        printf("Child collected successfully.\n");
        printf("Parent terminating...\n");
    }

    return 0;
}
```

Observation :

- Multiple child processes were successfully created using fork().
- wait() collected whichever child terminated first.
- waitpid() allowed the parent to wait for a specific child.
- A zombie process was created when the child terminated before the parent called wait().
- The zombie was visible with state Z and <defunct> using ps.
- After adding waitpid(), the child process was properly collected and the zombie was eliminated.

Result :

The experiment successfully demonstrated the difference between wait() and waitpid(). wait() waits for any terminated child, while waitpid() can synchronize with a specific child. A zombie process was successfully created and identified using Linux process-monitoring commands. The zombie was eliminated by using proper parent-child synchronization with waitpid().